ASSIGNMENT-2

Natural Language to Logic Circuit Generator

- **PROBLEM STATEMENT**

  Manually translating natural language requirements into digital logic is time-consuming and highly prone to human error, often resulting in flawed hardware designs. There is a critical need for an automated, AI-driven system that accurately parses plain English descriptions to directly generate, minimize, and verify Boolean expressions, truth tables, and gate-level circuits.

- **OBJECTIVE**

  To convert an English description of a logical situation into a Boolean expression and gate-level design using GenAI, then simulate and verify.

- **USING GEN AI FOR SIMPLIFICATION**

  AI PROMPT

  Convert the following English description of a logical situation into a Boolean expression and propose a gate-level logic circuit: 'A bank vault door opens (V=1) only if the manager's key is turned (M=1) AND either the guard's key is turned (G=1) OR the central security override is active (S=1), provided it is NOT outside business hours (H=1 means outside hours, so we require H=0).' Provide the final Boolean equation and a truth table.

  AI RESULT

Response: The conditions for the vault door to open can be mapped to the following Boolean expression: **V = M · (G + S) · H'**

Here is how the variables break down:

- **M = 1**: Manager's key is turned
- **G = 1**: Guard's key is turned
- **S = 1**: Central security override is active
- **H' (NOT H)**: It is NOT outside business hours (since H=1 means outside hours, H must be 0, so we invert it).

- ## Gate-Level Logic Circuit

  To build this circuit using standard logic gates, you will need:

  1. **OR Gate**: Connect inputs **G** and **S** to a 2-input OR gate. This outputs a 1 if either the guard's key or the security override is active.

  2. **NOT Gate (Inverter)**: Connect input **H** to a NOT gate. This outputs a 1 only during business hours (when H=0).

  3. **AND Gate**: Connect input **M**, the output of the OR gate, and the output of the NOT gate into a 3-input AND gate (or chain two 2-input AND gates together). The output of this AND gate is **V**.

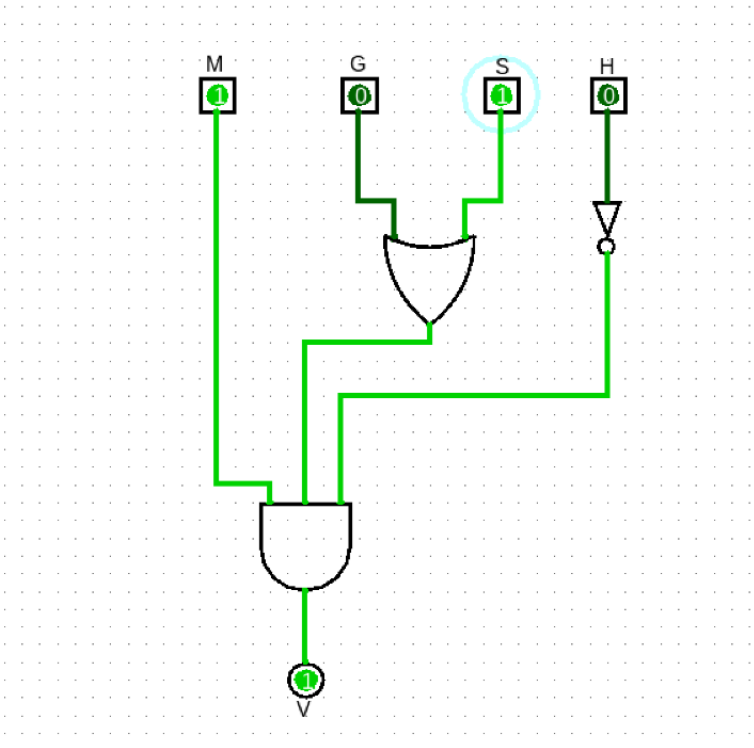
- ## Truth Table

  There are 4 input variables, resulting in 16 possible combinations. The vault (V) will only open (1) in the three specific scenarios where the manager is present, it is during business hours, and either the guard or the override is active.

M (Manager)	G (Guard)	S (Override)	H (Outside Hours)	V (Vault Opens)
0	0	0	0	**0**
0	0	0	1	**0**
0	0	1	0	**0**
0	0	1	1	**0**
0	1	0	0	**0**
0	1	0	1	**0**
0	1	1	0	**0**
0	1	1	1	**0**
1	0	0	0	**0**

1	0	0	1	**0**
1	0	1	0	**1**
1	0	1	1	**0**
1	1	0	0	**1**
1	1	0	1	**0**
1	1	1	0	**1**
1	1	1	1	**0**

- **CIRCUIT IMPLEMENTATION AND LOGISIM SIMULATION**



- ## Discussion and Verification (Reflection)

- **Truth Table Verification:**

  To simulate and verify the circuit, test various input combinations in Logisim. For the vault to open (V=1), M must be 1, H must be 0, and at least one of G or S must be 1. For example, if the manager turns their key (M=1), the guard is absent (G=0) but the override is active (S=1), during normal hours (H=0), the AND gate receives three 1s (M=1, G+S=1, H'=1) and outputs V=1. If it is outside business hours (H=1), the NOT gate outputs a 0, disabling the AND gate and forcing V=0 regardless of the keys. This matches the English description perfectly.

- **Reflection on GenAI's Output:**

  The GenAI successfully parsed complex natural language conditions into formal logic. It correctly identified that "either... or" maps to an OR gate, compulsory conditions ("only if", "and") map to an AND gate, and exclusionary clauses ("provided it is NOT") require a NOT gate. Translating "outside business hours" into an active-low condition (H') was handled seamlessly, bridging the gap between conversational rules and strict Boolean algebra.